Project Report

Project ID: 42 - Dataset: 8
Alessandro Lucca 10715182 - Simone Oggioni 10714793

Setup choices

Libraries used to perform the operations in the **data quality assessment** and the **data cleaning**:

- ➢ pandas
- ➢ numpy
- ➢ re
- ➢ json
- ➢ jaro from jaro-winkler
- ➢ ProfileReport from ydata_profiling

Pipeline implementation

First, we import the dataset
'Comune-di-Milano-Attivita-commerciali-di-media-e-grande-distribuzione.csv'

Then, we start from the **data quality assessment** in order to have a first understanding of the problems of the dataset.

Data quality assessment

An important assumption that we make is that we don't assume any knowledge about the ground-truth domain. As a consequence, **Accuracy** can't be evaluated. Also **Timeliness** can't be evaluated because there is not sufficient information to assess the currency and the volatility of the data.
As a result, at the dataset level the investigated quality dimensions are:

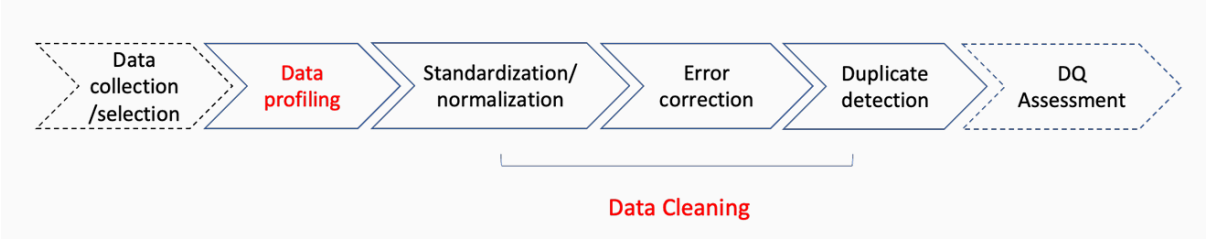
- **Completeness**
- **Consistency**

The completeness refers to the number of not missing values over the total. We have a completeness of 88.12 %.
For the consistency dimension we choose to define the following **consistency rule**:
'Superficie vendita' + 'Superficie altri usi' = 'Superficie totale'
The consistency of this rule is 15.81 %, this means that we will have to work on this during the cleaning pipeline.

To be aware of all the characteristics and problems of each feature we run a very useful and ready-to-use tool as the **ProfileReport from ydata_profiling**, to produce an HTML document with all the necessary information. In the ProfileReport there is also the (automatically computed) assessment of the distinctness and the percentage of missing values for each feature.

Data cleaning pipeline

Our **data cleaning pipeline** follows the optimal pipeline studied during lectures



The data cleaning pipeline is composed of:

1. **Data normalization and standardization**

    1.1 Datatype conversion

    1.2 Normalization

        1.2.1 “Settore Merceologico”

        1.2.2 “Ubicazione”

2. **Error correction**

    2.1 Missing values

    2.2 Inconsistencies

    2.3 Outliers handling

3. **Duplicate detection**

    3.1 Exact matching

    3.2 Similarity measures

Before producing a step-by-step description of the operations performed during these stages, it is important to describe our **reasoning** while dealing with this dataset.

Indeed, in our cleaning pipeline we have worked with the dataset to organize it as thoroughly as possible, assuming the information it contains to be accurate, and avoiding the imputation of arbitrary data. This is because the dataset may be enriched with additional information in the future, such as by someone verifying details on-site (e.g., checking which supermarket is at a particular address and adding that information).

In the end, we also add the sections regarding a new consistency rule, the reordering of columns and the saving of the cleaned dataset, before the results.

Data Normalization and Standardization

Datatype Conversion

During the preliminary data analysis, using the dtypes function in pandas, we identified some **inconsistencies** in the data types of certain columns. Specifically:

- The “Civico” column was of type float, but since in some cases the values can contain characters or non-integer values, it was deemed more appropriate to convert it to ‘object’ type.
- The “Superficie altri usi” column was of type float, even though it contains only integer values. For better data handling, it was converted to ‘Int64’, which allows for the correct representation of integer values, including missing data.

Normalization

In this step, we address two primary normalization issues in the dataset:

1. Normalization of “Settore Merceologico”
2. Normalization of “Ubicazione”

“Settore Merceologico”

The “Settore Merceologico” column contains different categories that need to be normalized: ‘alimentare’, ‘non alimentare’, ‘tabella speciale carburanti’, ‘tabella speciale monopolio’, ‘tabella speciale farmacie’.
The unique values in this column are derived from various sectors types, and we aim to create separate binary columns for each category. These new columns will indicate whether a given record belongs to a specific sector (‘1’ value) or not (‘0’ value).
After the transformation, we drop the original “Settore Merceologico” column to avoid redundancy, and we ensure that the new columns are of type Int64 to manage possible missing values correctly.

“Ubicazione”

The “Ubicazione” column contains detailed location information, including street types, names, house numbers, and additional data that need to be extracted and normalized.
We perform the following steps:

- Extracting and normalizing components: we use regular expressions and extract specific parts of the location, such as the “Tipo Via”, “Via”, “Civico”, “Codice Via”, “ZD”, “Isolato” and “Accesso”.
- Creating a temporary dataframe “TEMP_DF” to store these extracted values.
- Post-Extraction Handling: we convert some of the columns, such as “Codice Via”, “Isolato”, and “ZD” to the ‘Int64’ datatype, ensuring proper handling of missing values. Another normalization is replacing “LARGO” street type with its abbreviated form “LGO” for consistency: all the “Tipo Via” elements are in abbreviated form.

The following regular expressions are used for extraction:

- “Tipo Via”: Extracted from predefined abbreviations (e.g., ALZ, BST, CSO).

- “Via”: The name of the street is captured, considering possible apostrophes and special characters.
- “Civico”: Identified by “N.” or “num.” and normalized by removing leading zeros or handling complex formats (e.g., “006/a” becomes “6/a”).
- “Codice Via”, “ZD”, “Isolato”, “Accesso”: Identified by introductive words (e.g., “accesso:”, “isolato:”, “codvia:”, “(z.d. 1)”.

Error Correction

Missing values

We deal with **missing values** by analyzing the features one at a time.
We classify each type of Missing value as:

- **MCAR** (Missing Completely At Random)
- **MAR** (Missing At Random)
- **MNAR** (Missing Not At Random)

In the following table, we underline the type of missing values and the strategy adopted for each feature with missing values

Feature	Type of Missing Values	Solution
[‘alimentare’, ‘non alimentare’, ‘tabella speciale carburanti’, ‘tabella speciale monopolio’, ‘tabella speciale farmacie’]	MAR if we find the same ‘Insegna’ with the value specified, otherwise MCAR	Fill with the exact value if find the same ‘Insegna’ with the value specified, otherwise fill all with 0
Insegna	MCAR	Leave blank space
Civico	MAR	Extract from ‘Ubicazione’
Superficie altri usi	MAR	Extract by doing ‘Superficie totale’ - ‘Superficie vendita’
Isolato	MCAR	Leave blank space
Accesso	MCAR	Leave blank space

We end this part with missing values left for the features ‘Insegna’, ‘Isolato’, ‘Accesso’.

Inconsistencies

The **inconsistencies** primarily emerge from differences between the values extracted from 'Ubicazione' and those already present in the dataset. We assume that the values in 'Ubicazione' are the most accurate, so we prioritize them as our primary source. The values already present in the dataset are used as a backup in cases where 'Ubicazione' doesn't contain the specific fields. Regarding surface area consistency, we prioritize 'Superficie vendita' and calculate 'Superficie totale' by summing 'Superficie vendita' and 'Superficie altri usi.'

Here we present the table with the inconsistencies and how we solve them.

Inconsistency	How to solve the inconsistency
‘Tipo Via’ extracted from ‘Ubicazione’ and ‘Tipo Via’ already present in the dataset	Assumption: when not null the correct value of ‘Tipo Via’ is the one extracted from ‘Ubicazione’
‘Via’ extracted from ‘Ubicazione’ and ‘Via’ already present in the dataset	Assumption: when not null the correct value of ‘Via’ is the one extracted from ‘Ubicazione’
‘Civico’ extracted from ‘Ubicazione’ and ‘Civico’ already present in the dataset	Assumption: when not null the correct value of ‘Civico’ is the one extracted from ‘Ubicazione’
‘Codice Via’ extracted from ‘Ubicazione’ and ‘Codice Via’ already present in the dataset	Assumption: when not null the correct value of ‘Codice Via’ is the one extracted from ‘Ubicazione’
‘ZD’ extracted from ‘Ubicazione’ and ‘ZD’ already present in the dataset	Assumption: when not null the correct value of ‘ZD’ is the one extracted from ‘Ubicazione’
‘Superficie altri usi’ in certain cases is lower than zero	‘Superficie altri usi’ MUST BE greater or equal to zero
‘Superficie altri usi’ + ‘Superficie Vendita’ != ‘Superficie totale’	Assign ‘Superficie Totale’ to be equal to ‘Superficie altri usi’ + ‘Superficie Vendita’

Outlier handling

In this step, we examine the dataset for outliers in the “Superficie totale” column. The aim is to identify and analyze any **extreme values** that may indicate **potential data anomalies**.
Outliers are identified using the **Interquartile Range (IQR)** method.
The interquartile range is the middle half of the data that lies between the upper and lower quartiles. In other words, the interquartile range includes the 50% of data points that are above Q1 and below Q3.

The first point is to calculate Quartiles and IQR: Q1 is the lower quartile value (25%), Q3 is the higher quartile value (75%) and IQR is the difference Q3-Q1 (remaining 50%).
Then we define the bounds:

- Lower bound: Q1 - 1.5 × IQR
- Upper bound: Q3 + 1.5 × IQR

Data values that are outside this interval, are declared to be **outliers**.

To visually represent these outliers, we created a **boxplot** for “Superficie totale”, highlighting the extreme values in red. The resulting boxplot clearly visualize the presence of extreme values that fall outside the expected range.

The five most extreme outliers, sorted by “Superficie totale”, were further analyzed and some observations have been made:

- some of them represent well-known establishments with legitimate large surfaces.

- one of them raises some suspicion due to its significant “Superficie altri usi”.
- another appears also questionable due to its large “Superficie vendita”, though no additional contextual information is available (e.g., “Insegna”).

Without further data to validate or refute these values, all identified outliers have been retained as valid.

Duplicate detection

Exact Matching

In this part, we remove the **duplicated rows**, whose presence was found in the ProfileReport.

Similarity measures

There are two features that require a similarity check.

The first one is ‘Insegna’, because even at a first look at the dataset we could notice that there were different strings for the same ‘Insegna’. This difference mainly arises from the lack of a **standardized naming convention** for each 'Insegna'. For example, we encounter variations like 'max E co', 'max E co.', and 'max E co max mara,' all of which refer to the same brand. Therefore, it's necessary to map these names into a unified format.
The similarity detection process is:

1) Filter out common words (such as ‘supermercato’, ‘discount’, etc.) from the strings because these common words can cause two different ‘Insegna’ to result as similar.
2) Apply **Jaro similarity** (a string-based similarity function) to compute the similarity between the filtered strings and print as similar the strings that reach at least 0.85 as similarity score.
   Our choice is to use Jaro because it is good at handling typos and different business names. Also, it is less sensitive to the string length. A threshold of 0.85 is effective to filter out false positives as much as possible.
   Based on an empirical observation of the results, we map the similar strings in a common format.
3) Return at analyzing the original strings and apply **Jaccard similarity** (an item-based similarity function) to compute the similarity. In this case, we put a threshold of 0.5 since we want to output strings that have at least one word in common.
   This approach allows us to map other ‘Insegna’ into a common format.

The second feature to analyze is ‘Via’. Given that ‘Via’ has no common names and no typos, we just check for similarity by using the **Jaccard similarity**. Also here, a reconciliation of similar strings in a common format is performed based on the printed output.

Reiteration: new consistency checks

After cleaning the dataset, we perform a **final consistency check** to ensure the integrity of the data. The focus is on the relationships between specific fields, particularly “Via” should have consistent values for:

- “Tipo Via”
- “ZD”

The rule we are validating is “If rows share the same value for ‘Via’, they should also have the same values for ‘Tipo Via’ and ‘ZD’.

We found that, in reality, some exceptions exist:

1. A street name can correspond to one of a different “Tipo Via”
2. Streets typically fall within a single administrative zone (ZD) but they can also be on the border between two zones.

For this reason, we have controlled the few inconsistencies we found to confirm that they respected these two cases. So, we also consider these results as correct.

Reordering the columns

The final step is to organize the dataset for **readability** and **usability**. This involves:

1. Dropping unnecessary columns (e.g., “Ubicazione”)
2. Reordering the remaining columns in a logical sequence: first of all primary information, then geographic and administrative details, then metrics, and in the end category flags.
3. Renaming columns to improve clarity and standardize naming convention: capitalize their names, replacing underscores with spaces, using more descriptive labels.
4. Sorting rows by a key attribute for better organization (e.g., “Insegna”).

Saving the cleaned dataset

The **cleaned and organized dataset** is saved for future use or analysis.

Results

If we try to run again the consistency rule defined in the **data quality assessment** part, we obtain a score of 100 %, so the **consistency** check is passed.

It is important to underline the main improvements thanks to our data **cleaning pipeline**:

- It is possible to rely on correct data about the address, since all the inconsistencies are solved. This is crucial if the administrators of the dataset want to update it by inputting the missing values from ‘Insegna’, ‘Isolato’ or ‘Accesso’ (by Internet research or by physically research on the field)
- It is possible to rely on correct data about the surface, since all the missing values are removed and the inconsistencies are solved. This admits to being able to perform specific data analysis tasks in the future.
- It is possible to rely on correct data about the ‘Insegna’, since the similar ones have been reconciled to a common format. Specific data analysis tasks can be performed, for example grouping for each ‘Insegna’ is now possible.